## Presentation layer

One interface, two shells — identical analysis surface

TIER 1

### Tauri 2 desktop shell

Windows · macOS (Intel & Apple silicon) · Linux

### Web UI — React + TypeScript

Vite single-page app served by the shell or hosted

### Network views — Sigma.js

WebGL renderer with automatic 2D-canvas fallback

### Full RTL interface mirroring

Arabic and Urdu right-to-left UI, Cairo/Segoe fonts

JSON over localhost — the UI never touches Python directly

## Analysis engine

Local-first: nothing leaves the researcher's machine

TIER 2

### FastAPI application server

REST API over localhost; ships as a packaged sidecar with the desktop app

### SQLite project storage

Corpora, documents, vector cache, saved queries, provenance

### Background job runner

Chunked, cancellable bulk jobs with progress; one bulk job at a time

### Smart Troubleshooting

Built-in diagnostics for models, resources and engine health

in-process calls to model libraries — no subprocess per query

## NLP & AI layer

Deterministic NLP first; language models only where they earn their place

TIER 3

### spaCy pipelines × 5 languages

English, Arabic, Urdu, Hindi, Persian — POS, dependency, sentences

### CAMeL Tools (Arabic)

Morphology: root, pattern, lemma, stem; dialect & register identification

### Vector search — bge-m3

Multilingual embeddings via Ollama; cached locally in SQLite

### Grounded AI agent

27 registered tools; local Ollama or cloud OpenAI / Anthropic

## Student Mode

Optional classroom layer: the teacher's device acts as the classroom server.

- Read-only classroom site over the local network, served by a Caddy sidecar
- Concurrent seats capped by device and LM resources
- Anonymous audit log: every query + answer, no IPs, tokens or session IDs
- Students join by URL or QR code; browser only

All processing is local; the only optional network calls are user-configured AI providers (Ollama, OpenAI, Anthropic).